3.11 原码乘法的硬件逻辑实现

考纲定位：计算机组成原理 · 第 3 章 运算方法（3.11） | 重要度 ★★★★ | 常见考法：选择题（符号位处理、数值位乘法、与原码/补码关系）

一句话核心：原码乘法把符号与数值分开：符号位异或，数值位按无符号乘法算

一、分析

原码乘法可基于 3.10 无符号数乘法的硬件逻辑实现 的逻辑实现，只需多处理一位符号位

符号处理

- 结果符号 $= x_s \oplus y_s$
- 数值位 $= |x| \times |y|$
- 符号位不参与数值乘法

运算推导

设 $[X]_{原} = X_0.X_1X_2\cdots X_n$，$[Y]_{原} = Y_0.Y_1Y_2\cdots Y_n$，**其中** X_0 **和** Y_0 **分别为** $[X]_{原}$ **和** $[Y]_{原}$ **的符号位**

则 $[X]_{原} \times [Y]_{原} = [X \times Y]_{原} = X_0 \oplus Y_0 . ((0.X_1X_2\cdots X_n) \times (0.Y_1Y_2\cdots Y_n))$

乘积的符号位　　$|X|$　　$|Y|$

设 $x = -0.1101$，$y = 0.1011$，**计算** $[x \times y]_{原}$。

$[x]_{原} = 1.1101$，$[y]_{原} = 0.1011$，　$[x \times y]_{原} = 1 \oplus 0 \ . \ (|x| \times |y|)$

$$
\begin{aligned}
|x| \times |y| \ &= |x| \times 0.1011 \ = |x| \times (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}) \\
&= |x| \times 1 \times 2^{-1} + |x| \times 0 \times 2^{-2} + |x| \times 1 \times 2^{-3} + |x| \times 1 \times 2^{-4} \\
&= 2^{-1}(|x| \times 1 + |x| \times 0 \times 2^{-1} + |x| \times 1 \times 2^{-2} + |x| \times 1 \times 2^{-3}) \\
&= 2^{-1}(|x| \times 1 + 2^{-1}(|x| \times 0 + |x| \times 1 \times 2^{-1} + |x| \times 1 \times 2^{-2})) \\
&= 2^{-1}(|x| \times 1 + 2^{-1}(|x| \times 0 + 2^{-1}(|x| \times 1 + |x| \times 1 \times 2^{-1}))) \\
&= 2^{-1}(|x| \times 1 + 2^{-1}(|x| \times 0 + 2^{-1}(|x| \times 1 + 2^{-1}(|x| \times 1 + 0))))
\end{aligned}
$$

可参看上节课（无符号数乘法运算的硬件逻辑实现）这部分的推导

提炼

- 原码表示下，符号位与数值位独立
- 乘法规则：符号异或，绝对值相乘
- 结果原码 = 结果符号 + 数值乘积

易错

- 原码乘法的数值位是无符号乘法，不要按补码算
- 符号位异或后要放回结果最高位
- 0 的符号要单独处理；原码有 +0 和 -0

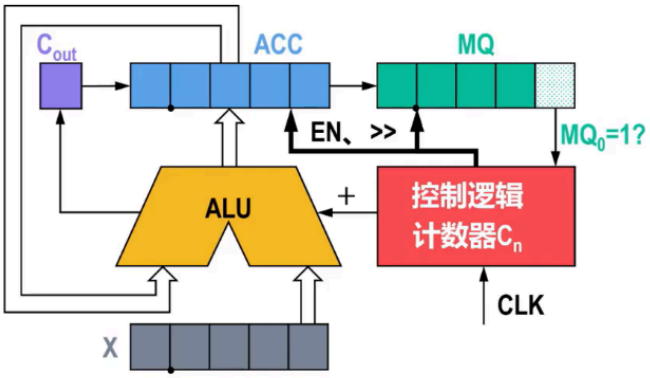
二、硬件逻辑实现

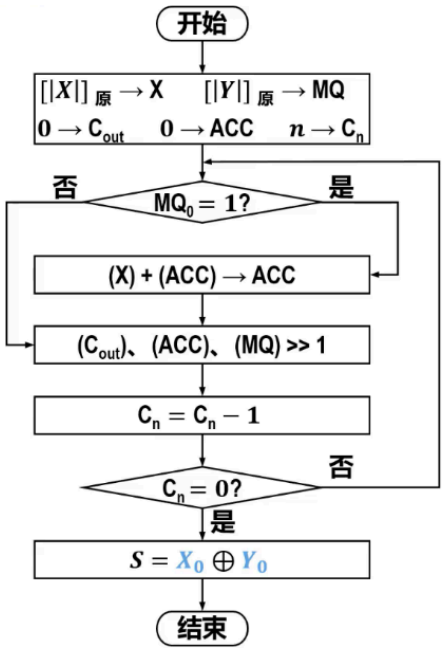
设 $x=-0.1101$，$y=0.1011$，计算 $[x\times y]_{原}$。

$[x]_{原}=1.1101$，$[y]_{原}=0.1011$，$[|x|]_{原}=0.1101$，$[|y|]_{原}=0.1011$

$[x\times y]_{原}=1\oplus 0\ .\ (|x|\times|y|)$

$|x|\times|y|=2^{-1}(|x|\times 1+2^{-1}(|x|\times 0+2^{-1}(|x|\times 1+2^{-1}(|x|\times 1+0))))$





结构

- 符号位异或门产生结果符号
- 数值位送入无符号乘法器
- 乘积数值位与符号位拼接成原码结果

过程

设 $x=-0.1101$，$y=0.1011$，计算 $[x\times y]_{原}$。

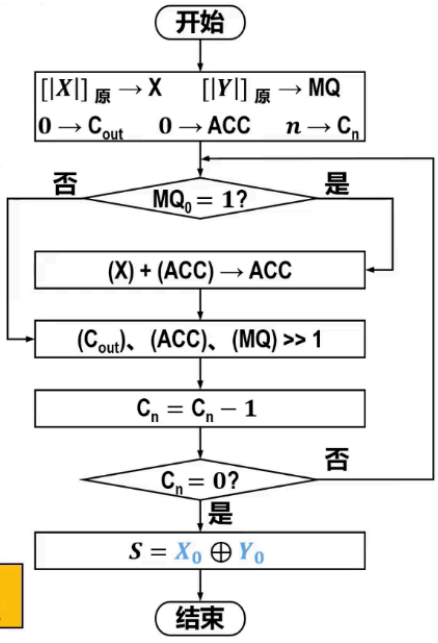
$[x]_{原}=1.1101$，$[y]_{原}=0.1011$，$[|x|]_{原}=0.1101$，$[|y|]_{原}=0.1011$

$[x\times y]_{原}=1\oplus 0\ .\ (|x|\times|y|)\ =1.10001111$

$|x|\times|y|=2^{-1}(|x|\times 1+2^{-1}(|x|\times 0+2^{-1}(|x|\times 1+2^{-1}(|x|\times 1+0))))=0.10001111$

加数 被乘数	进位 信号	被加数、和 乘积高位	乘数 乘积低位
X	C_{out}	ACC	MQ
初始状态	01101	0	00000	01011
(X) + (ACC) → ACC	01101	0	01101	01011
(Cout)、(ACC)、(MQ) >> 1	01101	0	00110	10101
(X) + (ACC) → ACC	01101	0	10011	10101
(Cout)、(ACC)、(MQ) >> 1	01101	0	01001	11010
(Cout)、(ACC)、(MQ) >> 1	01101	0	00100	11101
(X) + (ACC) → ACC	01101	0	10001	11101
(Cout)、(ACC)、(MQ) >> 1	01101	0	01000	11110

舍弃乘数y的符号位



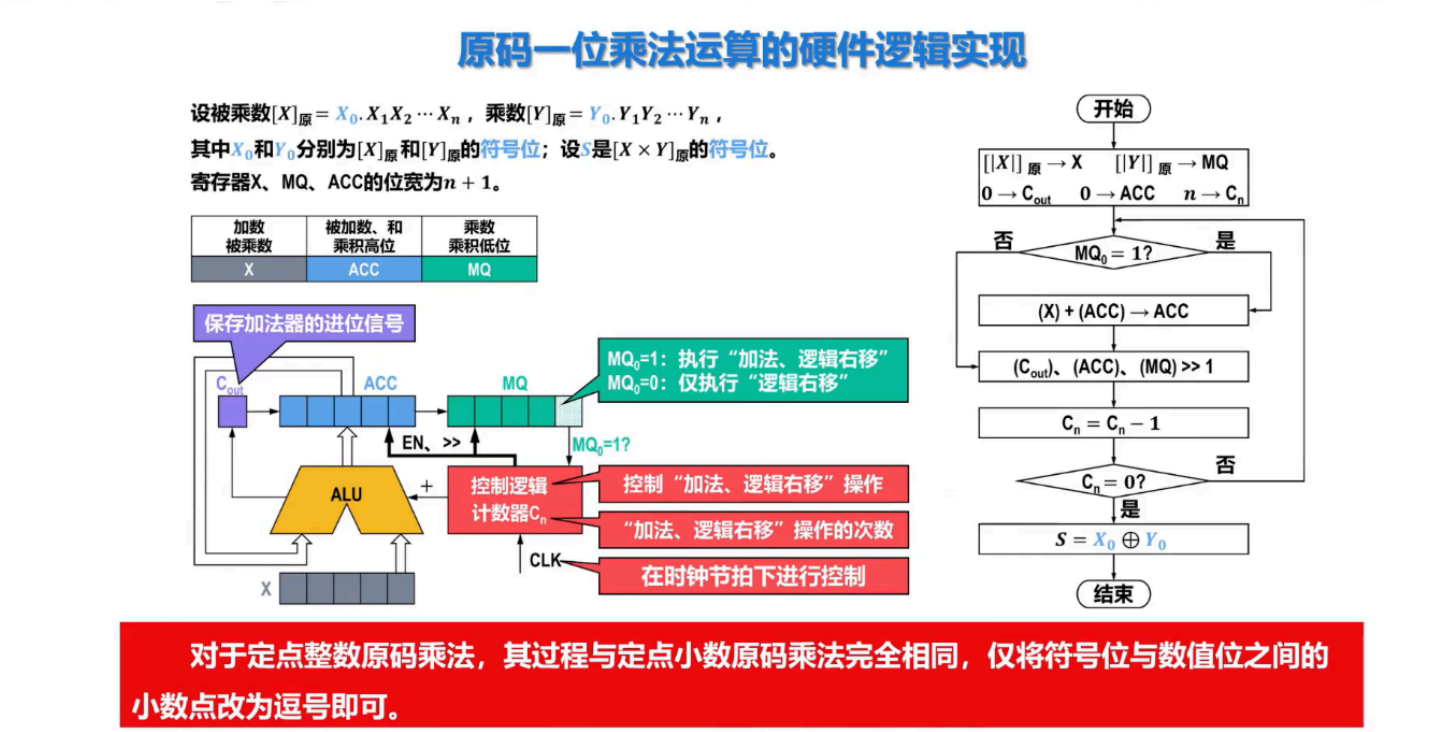
步骤

1. 取出两个操作数的符号位，异或得到结果符号
2. 取绝对值，送入无符号乘法器
3. 数值位相乘，得到 $2n$ 位乘积
4. 拼接符号位，得到原码结果

易错

- 绝对值运算不是“取反加 1”；原码的数值位直接取，符号位单独处理
- 结果位数是 $2n$ 位（数值位）+ 1 位符号
- 原码乘法不适合直接用于补码数据；补码要用 Booth 算法

三、总结



项目	原码乘法
符号	异或
数值	无符号乘法
结果	原码
硬件	符号异或 + 无符号乘法器
适用	原码表示的数据

易错

- 原码乘法与补码乘法不能混用
- 符号位不参与数值运算，但结果符号必须单独产生
- 绝对值相乘可能产生 $2n$ 位结果，不能截断

知识关联：无符号乘法（3.10 无符号数乘法的硬件逻辑实现）；Booth 算法（3.12 补码乘法的硬件逻辑实现——Booth算法）；原码表示（第 2 章）